MÓDULO FORMATIVO AVANZADO • INGENIERÍA DE DATOS Y ADMINISTRACIÓN DE SGBD

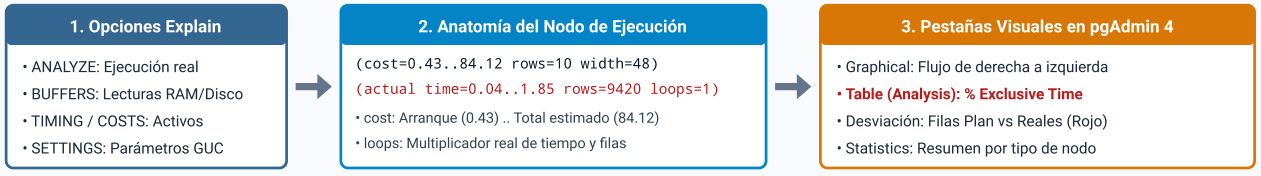
Optimización de Consultas y Troubleshooting en PostgreSQL

Guía de referencia técnica para el análisis de planes de ejecución con pgAdmin 4, refactorización de SQL, gestión de memoria, mantenimiento de índices y resolución de bloqueos transaccionales.

1. Metodología de Análisis de Consultas en pgAdmin 4

El optimizador de consultas de PostgreSQL es un motor basado en costes (*Cost-Based Optimizer* o CBO) que evalúa múltiples rutas de acceso utilizando las estadísticas almacenadas en el catálogo `pg_statistic`. Para auditar una consulta en **pgAdmin 4** desde el *Query Tool*, disponemos de dos modos: **Explain (F7)**, que proyecta el plan estimado sin ejecutar la sentencia, y **Explain Analyze (Shift+F7)**, que ejecuta realmente la consulta e instrumenta tiempos reales e I/O de bloques.

Figura 1.1: Flujo de Auditoría Visual en pgAdmin 4 y Anatomía de Métricas del Nodo



2. Diccionario de Lectura de Planes de Ejecución

Un plan en texto se lee desde el nodo más indentado (hoja interna) hacia la raíz superior. En la pestaña gráfica de pgAdmin, se lee desde la derecha hacia la izquierda. La tabla siguiente resume cómo interpretar cada indicador:

Métrica / Indicador	Significado Técnico en PostgreSQL	Patrón de Alerta y Diagnóstico Pedagógico
`cost=A..B`	Unidades arbitrarias de coste relativas a `seq_page_cost=1.0`. `A` es coste de inicio; `B` es coste total del nodo.	Un coste de arranque alto indica un operador bloqueante (ej. `Sort` o `HashAggregate`) que debe procesar todo antes de emitir la 1ª fila.
`rows` (Plan vs Actual)	Comparación entre la cardinalidad estimada por el planificador y las filas reales devueltas en ejecución.	Si difieren en más de 10x, el optimizador eligió el algoritmo equivocado (ej. `Nested Loop` en vez de `Hash Join`). Ejecutar `ANALYZE`.
`loops=N`	Número de veces que el nodo hijo ha sido ejecutado por su nodo padre (típico en bucles anidados).	El `actual time` reportado es el promedio por iteración. **Tiempo real consumido = actual time × loops**.
`Buffers: shared hit`	Bloques de 8 KB leídos directamente desde la memoria RAM del motor (`shared_buffers`).	Es el comportamiento ideal. Un ratio de acierto en caché saludable en cargas OLTP debe superar el 98%-99%.
`Buffers: shared read`	Bloques solicitados al caché del Sistema Operativo o leídos físicamente desde el disco de almacenamiento.	Valores altos junto con tiempos elevados revelan cuellos de botella de lectura por falta de índices o memoria insuficiente.
`Buffers: temp read/written`	Volcado de datos intermedios a disco temporal (*Spill to Disk*) en el directorio `pgsql_tmp`.	Indica que una ordenación o tabla hash superó el límite de `work_mem` de la sesión. Penaliza drásticamente la latencia.

Precaución en Producción con EXPLAIN ANALYZE

Dado que `EXPLAIN ANALYZE` ejecuta realmente la consulta para medir tiempos reales, si se aplica sobre

3. Catálogo de Nodos de Escaneo y Algoritmos de Unión (Joins)

Comprender por qué el optimizador selecciona un operador concreto permite a los alumnos diagnosticar si una consulta sufre por ausencia de índices, exceso de lecturas aleatorias o desajustes en los parámetros de coste.

Operador del Plan	Mecanismo Interno de Funcionamiento	Cuándo es Óptimo / Cuándo Optimizar
Seq Scan Escaneo Secuencial	Lee todos los bloques de la tabla secuencialmente en disco/RAM y evalúa el predicado fila por fila (`Filter: ...`).	Óptimo si la consulta recupera >15-20% de la tabla o tablas muy pequeñas. Crítico si filtra pocas filas en tablas de millones de registros.
Index Scan Índice + Tabla	Recorre el árbol B-Tree y por cada puntero (`ctid`) salta aleatoriamente al bloque de la tabla (*Heap*) para comprobar visibilidad MVCC.	Ideal para alta selectividad (pocas filas). Si devuelve miles de filas desordenadas en disco, degrada por exceso de I/O aleatorio.
Index Only Scan Máxima Eficiencia	Resuelve la consulta leyendo únicamente las páginas del índice sin visitar la tabla base, apoyándose en el *Visibility Map*.	Requiere que todas las columnas del `SELECT` estén en el índice (cláusula `INCLUDE`) y que `VACUUM` mantenga limpio el mapa de visibilidad.
Bitmap Index / Heap Scan	Construye en memoria un mapa de bits con las páginas que contienen coincidencias, las ordena por dirección física y lee la tabla secuencialmente.	Excelente para selectividad media o para combinar múltiples índices mediante operaciones lógicas `BitmapAnd` / `BitmapOr` .
Nested Loop Join	Por cada fila de la relación externa, busca coincidencias en la relación interna (mediante índice o escaneo completo).	Muy rápido si la tabla externa devuelve pocas filas y la interna posee un índice en la clave de unión. Letal sin índice interno.
Hash Join	Carga la tabla más pequeña en una tabla Hash en memoria (`work_mem`) y escanea la tabla grande buscando colisiones.	El más eficiente para unir conjuntos medianos o grandes no ordenados. Vigilar `Batches > 1` (indica desbordamiento a disco).
Merge Join	Avanza en paralelo sobre ambas relaciones previamente ordenadas por la columna de unión.	Óptimo para conjuntos gigantescos cuando ambas entradas ya vienen ordenadas desde un `Index Scan` previo.

4. Laboratorio de Refactorización de Consultas (Casos Prácticos)

Caso Práctico 1: Ruptura de SARGability e Índice de Cobertura (Covering Index)

Problema: Aplicar una función sobre una columna indexada impide el uso del índice B-Tree estándar, forzando un `Seq Scan` que descarta el 99% de la tabla.

```
-- ❌ CONSULTA INEFICIENTE (Seq Scan: 485.20 ms | Buffers: shared read=18420)
SELECT id_pedido, total, estado FROM pedidos
WHERE EXTRACT(YEAR FROM fecha_creacion) = 2026 AND cliente_id = 4092;

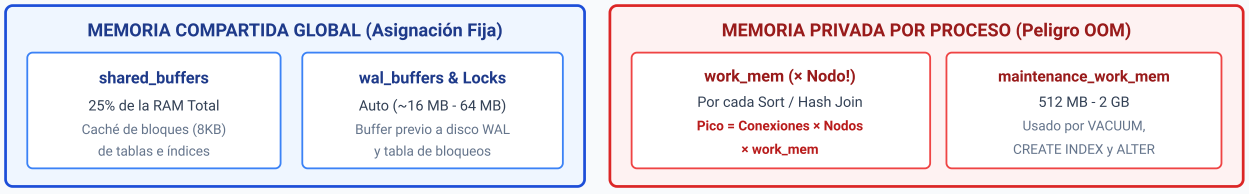
-- ✅ SOLUCIÓN: Predicado de rango (SARGable) + Índice Compuesto con INCLUDE
CREATE INDEX idx_pedidos_cliente_fecha ON pedidos (cliente_id, fecha_creacion) INCLUDE
(total, estado);

SELECT id_pedido, total, estado FROM pedidos
WHERE cliente_id = 4092 AND fecha_creacion >= '2026-01-01' AND fecha_creacion < '2027-
01-01';
-- Resultado: Index Only Scan using idx_pedidos_cliente_fecha (0.08 ms | Buffers: shared
hit=4 | Heap Fetches: 0)
```

5. Diagnóstico y Resolución de Cuellos de Botella en Memoria RAM

PostgreSQL divide su arquitectura de memoria en dos áreas críticas: una zona global compartida entre todos los procesos del servidor y una zona privada asignada dinámicamente por cada operación dentro de cada conexión activa. El error más común en producción es elevar `work_mem` globalmente sin considerar la concurrencia máxima, disparando el *OOM Killer* del kernel Linux.

Figura 5.1: Arquitectura de Memoria de PostgreSQL y Riesgo de Saturación por Concurrencia



6. Fórmulas de Dimensionamiento y Prevención de Volcado a Disco

Parámetro GUC	Regla de Cálculo Recomendada	Síntoma de Mal Ajuste y Estrategia de Corrección
`shared_buffers`	`25%` a `40%` de RAM total	Si es muy bajo, incrementa `Buffers: shared read`. No superar el 40% porque PostgreSQL confía también en el Page Cache del kernel Linux (evitar doble buffering).
`work_mem`	`(RAM Libre) / (max_connections × 3)`	Si aparece `External Merge Disk` en un `Sort` o `Batches > 1` en un `Hash`, subirlo a nivel de sesión (`SET LOCAL work_mem = '128MB';`) solo para reportes pesados.
`effective_cache_size`	`75%` de la RAM total	No reserva memoria física. Informa al optimizador de cuánta RAM total (`shared_buffers` + caché del SO) está disponible para favorecer `Index Scans` frente a `Seq Scans`.
`random_page_cost`	`1.1` en SSD/NVMe (Defecto: `4.0`)	El valor por defecto (4.0) asume discos mecánicos HDD. En almacenamiento SSD/Cloud, bajarlo a `1.1` evita que el planificador descarte índices válidos injustificadamente.

7. Auditoría de Lecturas en Caché y Archivos Temporales en Vivo

Script de Diagnóstico: Cache Hit Ratio Global y Detección de Spills a Disco

Ejecute esta consulta sobre `pg_stat_database` para verificar si la base de datos sufre lecturas físicas excesivas o si las consultas están creando ficheros temporales en disco por falta de `work_mem`:

```
SELECT datname AS base_datos,
       ROUND(100.0 * blks_hit / NULLIF(blks_hit + blks_read, 0), 2) AS
cache_hit_ratio_pct,
       temp_files AS archivos_temporales_creados,
       pg_size_pretty(temp_bytes) AS volumen_volcado_a_disco
FROM pg_stat_database
WHERE datname = current_database();
-- Meta OLTP: cache_hit_ratio_pct > 99.00% | Si temp_bytes crece rápido, auditar queries
con pg_stat_statements.
```

8. Optimización de Índices y Mantenimiento del Motor (Bloat)

Seleccionar el tipo de índice adecuado reduce tanto la huella de almacenamiento como el coste de escritura. Sin embargo, debido al modelo MVCC de PostgreSQL, cada `UPDATE` o `DELETE` genera tuplas muertas (*Dead Tuples*) que inflan los índices (*Bloat*) y degradan la velocidad de escaneo.

1. Índices Parciales (Partial Indexes)

Indexan únicamente el subconjunto de filas que cumple una condición frecuente, reduciendo hasta un 90% el tamaño del árbol B-Tree:
`CREATE INDEX idx_ped_pend ON pedidos(fecha) WHERE estado = 'PENDIENTE';`

2. Índices GIN y BRIN

GIN: Imprescindible para búsquedas internas en columnas `JSONB`, arrays y *Full-Text Search*.
BRIN: Guarda solo rangos mín/máx por bloques físicos. Ocupa un 99% menos que B-Tree en series temporales correlacionadas.

3. Eliminación de Índices Sin Uso

Cada índice ralentiza los `INSERT/UPDATE` e impide actualizaciones rápidas *HOT (Heap-Only Tuple)*. Los índices con `idx_scan = 0` tras un ciclo completo de negocio deben eliminarse.

4. Mitigación de Bloat sin Bloqueos

Cuando un índice acumula más del 30% de páginas vacías por borrados masivos, reconstruirlo en caliente sin bloquear lecturas ni escrituras:
`REINDEX INDEX CONCURRENTLY idx_nombre;`

Script de Auditoría: Detección de Índices Inútiles y Tablas con Acumulación de Dead Tuples

```
-- 1. Localizar índices grandes que nunca son utilizados por el optimizador (excluyendo
claves únicas)
SELECT schemaname, relname AS tabla, indexrelname AS indice,
       pg_size_pretty(pg_relation_size(indexrelid)) AS tamano_indice, idx_scan AS
veces_usado
FROM pg_stat_user_indexes JOIN pg_index USING (indexrelid)
WHERE idx_scan = 0 AND NOT indisunique ORDER BY pg_relation_size(indexrelid) DESC LIMIT
10;

-- 2. Detectar tablas que requieren ajuste urgente de Autovacuum por exceso de tuplas
muertas
SELECT relname AS tabla, n_live_tup AS filas_vivas, n_dead_tup AS filas_muertas,
       ROUND(100.0 * n_dead_tup / NULLIF(n_live_tup + n_dead_tup, 0), 1) AS pct_muertas,
last_autovacuum
FROM pg_stat_user_tables WHERE n_dead_tup > 10000 ORDER BY pct_muertas DESC;
```

9. Saturación del WAL (Write-Ahead Log) y Tormentas de Checkpoints

Toda modificación se escribe primero secuencialmente en el registro **WAL** antes de confirmar la transacción, y posteriormente el proceso *Checkpointer* vuelca los bloques sucios de `shared_buffers` a los ficheros de datos definitivos.

Diagnóstico y Ajuste de Checkpoints Forzados (I/O Spikes)

- **Síntoma en Logs y Vistas:** Si en `pg_stat_bgwriter` (o `pg_stat_checkpointer` en PG 17+) la columna `checkpoints_req` (forzados por llenado de WAL) supera a `checkpoints_timed` (programados por tiempo), el servidor sufre picos severos de escritura en disco que congelan las lecturas.
- **Ajuste en `postgresql.conf`:** Incrementar `max_wal_size = 16GB` (o superior según carga de escritura), establecer `checkpoint_timeout = 15min` a `30min`, y fijar `checkpoint_completion_target = 0.9` para repartir la escritura de bloques sucios de forma suave durante el 90% del intervalo.
- **Compresión y Slots Huérfanos:** Activar `wal_compression = zstd` (o `lz4`) para reducir el volumen de *Full Page Writes* tras cada checkpoint. Auditar diariamente `SELECT slot_name, active, pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) FROM` [illegible] ya que un slot inactivo retiene archivos WAL indefinidamente hasta llenar el disco.

10. Gestión de Bloqueos de Transacciones e Interbloqueos (Deadlocks)

En PostgreSQL, las lecturas nunca bloquean a las escrituras y las escrituras nunca bloquean a las lecturas gracias a MVCC. Sin embargo, dos escrituras concurrentes sobre la misma fila, o un comando DDL (como `ALTER TABLE`) que solicita un bloqueo `AccessExclusiveLock` detrás de una transacción abierta (`idle in transaction`), generan un efecto dominó (*Lock Queue*) que paraliza toda la aplicación.

Script de Emergencia: Árbol de Bloqueos en Vivo (Quién bloquea a Quién)

Utiliza la función nativa `pg_blocking_pids()` para identificar instantáneamente la sesión raíz que retiene el cerrojo y el tiempo exacto que lleva bloqueando al resto de conexiones:

```
SELECT blocked.pid AS pid_bloqueado,
       blocked.usename AS usuario_bloqueado,
       now() - blocked.query_start AS tiempo_esperando,
       blocked.wait_event_type || ':' || blocked.wait_event AS evento_espera,
        left(blocked.query, 60) AS consulta_en_espera,
       blocking.pid AS pid_bloqueador_raiz,
       blocking.state AS estado_bloqueador,
       now() - blocking.xact_start AS duracion_tx_bloqueadora,
       left(blocking.query, 60) AS ultima_query_del_bloqueador
FROM pg_stat_activity AS blocked
JOIN pg_stat_activity AS blocking ON blocking.pid = ANY(pg_blocking_pids(blocked.pid))
WHERE cardinality(pg_blocking_pids(blocked.pid)) > 0
ORDER BY tiempo_esperando DESC;
```

11. Resolución de Interbloqueos (Deadlocks) y Timeouts Defensivos

Un **Deadlock** ocurre cuando la Transacción A bloquea la Fila 1 y espera por la Fila 2, mientras la Transacción B ya bloqueó la Fila 2 y espera por la Fila 1. Tras expirar `deadlock_timeout` (por defecto `1s`), PostgreSQL detecta el ciclo en el grafo de espera y aborta una de las transacciones con el error `40P01: deadlock detected`.

Mecanismo / Configuración	Valor Recomendado en Producción	Propósito Operativo y Prevención de Caídas
`lock_timeout`	`2s` a `5s` (en migraciones DDL)	Aborta una sentencia si no consigue adquirir el cerrojo en ese tiempo, evitando que un `ALTER TABLE` encole cientos de `SELECT` detrás de él.
`idle_in_transaction_session_timeout`	`30s` a `60s`	Cierra automáticamente sesiones que ejecutaron `BEGIN;` y quedaron colgadas por fallos en el código de la aplicación, liberando cerrojos y permitiendo actuar a `VACUUM`.
`statement_timeout`	`15s` a `60s` (OLTP Web)	Evita que consultas mal construidas consuman el 100% de CPU y memoria durante horas.
`log_lock_waits`	`on`	Registra en el log del servidor cualquier espera de bloqueo que supere `deadlock_timeout`, clave para auditoría forense post-incidente.
Ordenación Determinista	Patrón de Código SQL	Ordenar siempre los IDs antes de actualizar en lote (ej. `SELECT ... ORDER BY id FOR UPDATE`) elimina matemáticamente los Deadlocks.

Guía de Diseño Instruccional: Estructura Propuesta para la Unidad del Curso